

Table of Contents

1 Dossier técnico de `xtendoo-pos-conventional`

Repositorio analizado: `odoo/custom/src/xtendoo-pos-conventional` **Fecha del análisis:** 2026-05-14 **Ámbito:** módulos POS convencionales para Odoo 19
Documento fuente canónico: este fichero Markdown **Artefactos derivados recomendados:** PDF y DOCX generados desde este fichero

1.1 1. Resumen ejecutivo

`xtendoo-pos-conventional` implementa una **capa completa de TPV convencional/no táctil** sobre Odoo 19. No es un único módulo: es una **suite de 12 módulos** que transforma el flujo estándar del POS táctil en un flujo de **backoffice guiado por formularios, listas, popups OWL, escáner de códigos, cobro asistido, impresión automática, cierres de caja y control por PIN.**

1.1.1 Qué hace funcionalmente

- Permite operar un TPV desde backend en modo **no táctil**.
- Reutiliza `pos.order`, `pos.session`, `pos.payment`, `pos.config` y `sale.order`.
- Añade **sesión asistida**: apertura, caja inicial, movimientos de efectivo, cierre y arrastre de saldo.
- Añade **cobro guiado**: efectivo, tarjeta, mezcla de pagos, cálculo de cambio y reimpresión.
- Añade **escaneo de productos por código de barras** desde el formulario de pedido backend.
- Añade **tickets/recibos personalizados** y **factura simplificada 80mm**.
- Añade **devoluciones filtradas por caja**.
- Añade **integración con pedidos de venta / albarán**.
- Añade **restricción de cajas por usuario** y autenticación por **PIN POS**.

1.1.2 Hallazgos arquitectónicos importantes

1. **pos_conventional_core es el agregador principal** y depende del resto de módulos. Esto es importante porque parte de la lógica usa la cadena `super()` de forma deliberada, especialmente en `get_order_receipt_data`.
 2. El flujo está repartido entre:
 - o modelos Python,
 - o wizards transitorios,
 - o client actions,
 - o popups OWL,
 - o parches JS del backend,
 - o reports 80mm.
 3. El diseño está orientado a un **flujo backend-first**, no a la pantalla POS clásica.
 4. Hay bastante cuidado con casos complejos de Odoo 19:
 - o recomputación OWL en formularios one2many,
 - o multicompañía,
 - o impresión en segundo plano,
 - o consistencia de impuestos,
 - o cambios de pantalla con `beforeLeave`.
-

1.2 2. Inventario completo de módulos

Módulo	Rol principal	Dependencias directas	Superficie principal
pos_conventional_cash_calculator	Calculadora de billetes/monedas	point_of_sale, sale, mail	Wizard backend
pos_conventional_cash_drawer	Apertura de cajón desde flujo convencional	pos_conventional_payment_wizard, xtendoo_cash_drawer	Backend + JS
pos_conventional_config_user_filter	Filtrado de cajas POS por usuario	point_of_sale	Seguridad + controlador HTTP
pos_conventional_core	Núcleo agregador del	point_of_sale, sale, mail +	Backend + JS + vistas

Módulo	Rol principal	Dependencias directas	Superficie principal
	flujo convencional	casi toda la suite	
pos_conventional_order_barcode	Alta de líneas por escáner/código	point_of_sale, sale, mail	Modelo + JS formulario
pos_conventional_payment_wizard	Cobro guiado y popup de pagos	point_of_sale, sale, mail	Wizards + JS
pos_conventional_picking_integration	Conversión a venta/albarán	point_of_sale, stock, sale	Backend + report
pos_conventional_receipt	Personalización de ticket POS	point_of_sale, ll0n_es_pos	POS frontend assets
pos_conventional_receipt_custom	Factura simplificada, email y datos extra de ticket	point_of_sale, sale, mail	Backend + report
pos_conventional_returns	Flujo de devoluciones convencional	pos_conventional_core	Backend + JS
pos_conventional_sale_integration	Integración analítica con ventas	point_of_sale, pos_conventional_picking_integration, sale	Backend + reporting
pos_conventional_session_management	Apertura/cierre y control de caja	point_of_sale, sale, mail, pos_conventional_cash_calculator	Backend + OWL popups
pos_conventional_users_pin	PIN de usuario POS	point_of_sale, pos_conventional_session_management	Backend + wizard

1.2.1 Relación de dependencia global

```
pos_conventional_core
├── pos_conventional_receipt
├── pos_conventional_cash_calculator
├── pos_conventional_config_user_filter
├── pos_conventional_order_barcode
├── pos_conventional_payment_wizard
├── pos_conventional_picking_integration
├── pos_conventional_receipt_custom
├── pos_conventional_sale_integration
├── pos_conventional_session_management
└── pos_conventional_users_pin
```

```
pos_conventional_returns -> pos_conventional_core
pos_conventional_cash_drawer -> pos_conventional_payment_wizard +
xtendoo_cash_drawer
```

1.2.2 Lectura arquitectónica

- core no es “un módulo más”: actúa como **composición final** del sistema.
 - receipt toca el POS frontend clásico (point_of_sale._assets_pos), mientras que la mayoría del resto opera en web.assets_backend.
 - users_pin y session_management alteran el punto de entrada de apertura de caja.
 - payment_wizard y core cooperan para la validación final y la navegación a “nuevo pedido”.
-

1.3 3. Estructura técnica del repositorio analizado

1.3.1 Distribución observada

- **Python backend**: modelos heredados, wizards, reports, controladores HTTP.
- **XML**: vistas, acciones, assets, templates, seguridad.
- **JS/OWL**: controladores de lista/formulario, popups, client actions, parches del action service.
- **Tests**: backend Python y frontend JS/HOOT.

1.3.2 Dónde está la lógica más crítica

1. pos_conventional_core/models/
2. pos_conventional_payment_wizard/models/ y wizard/
3. pos_conventional_session_management/models/ y wizard/

- 4. `pos_conventional_order_barcode/models/` y `static/src/js/`
 - 5. `pos_conventional_receipt_custom/models/`
-

1.4 4. Arquitectura funcional end-to-end

1.5 4.1 Flujo macro

Config POS no táctil



Abrir caja (``pos.config.open_ui``)



Crear o recuperar ``pos.session``



Si estado = `opening_control` → PIN / popup apertura



Lista de pedidos POS de la caja actual



Nuevo pedido backend (``pos.order`` form)



Añadir líneas manualmente o por barcode



Recalcular precios, impuestos, costes y total



Cobro guiado / albarán / devolución



Validar + facturar + imprimir + siguiente pedido



Movimientos de caja / cierre de sesión

1.6 4.2 Entidades dominantes

- `pos.config`: activa el modo convencional y gobierna el comportamiento global.
 - `pos.session`: ciclo de caja.
 - `pos.order`: documento operacional central.
 - `pos.order.line`: líneas, impuestos, coste total y margen.
 - `pos.payment` / `pos.payment.method`: pagos y cambio.
 - `sale.order`: usado cuando el pedido se transforma en venta/albarán.
 - `account.move`: factura simplificada / impresión.
-

1.7 5. Módulo a módulo: análisis profundo

1.8 5.1 pos_conventional_core

1.8.1 Objetivo

Es el **núcleo funcional y de orquestación** del TPV convencional. Define el comportamiento no táctil, endurece la completitud de pedidos, recalcula totales en backend, compone la navegación y actúa como agregador de recibos y acciones cliente.

1.8.2 Dependencias destacadas

Depende de casi toda la suite. Esto le permite:

- usar `super()` para extender recibos,
- apoyarse en `session_management`,
- lanzar acciones de impresión,
- convivir con `order_barcode`, `payment_wizard`, `users_pin`, etc.

1.8.3 Backend clave

1.8.3.1 `models/pos_config.py`

Responsabilidades:

- campo `pos_non_touch`
- campo `default_partner_id`
- `_get_or_create_non_touch_session()`
- override de `open_ui()`
- redirección a lista de pedidos de la caja

Código importante:

```
if self.pos_non_touch:
    session = self._get_or_create_non_touch_session()
    if session.state == "opening_control":
        action = self._get_non_touch_opening_action(session)
    if session.state in ["opened", "closing_control"]:
        return self._redirect_to_pos_orders(session)
```

Interpretación:

- `open_ui()` deja de abrir el POS clásico cuando la caja es convencional.
- El punto de entrada natural pasa a ser la **lista de pedidos del backend**.

- El método `_get_non_touch_opening_action()` puede ser redefinido por otros módulos, especialmente `pos_conventional_users_pin`.

1.8.3.2 `models/pos_order.py`

Responsabilidades más importantes:

- ribbon visual con método de pago
- `amount_untaxed`
- validación fuerte de completitud en borrador
- `unlink()` seguro en multicompañía
- `default_get()` con sesión activa, tarifa y cliente por defecto
- `create()` con asignación automática de sesión abierta convencional
- cálculo de líneas con `pricelist` y descuento real
- recálculo de líneas al cambiar cliente/tarifa
- validación + facturación + navegación postventa
- composición de datos de recibo para frontend/printing

1.8.3.2.1 Regla de completitud

Solo en cajas no táctiles y pedidos en borrador:

- debe haber cliente,
- debe haber tarifa,
- debe haber líneas.

Esto evita “pedidos huérfanos” muy típicos cuando se trabaja desde formulario backend.

Código importante:

```
if order.state == "draft" and order.config_id.pos_non_touch:
    if not order.partner_id:
        raise ValidationError(...)
    if not order.pricelist_id:
        raise ValidationError(...)
    if not order.lines:
        raise ValidationError(...)
```

1.8.3.2.2 Asignación automática de sesión

Si se crea un pedido sin `session_id`, el módulo intenta usar:

1. la sesión abierta del usuario actual en modo convencional,
2. si no existe, la última sesión abierta convencional disponible.

Esto reduce errores operativos y hace más natural el flujo “nuevo pedido”.

1.8.3.2.3 Política de precios

`_prepare_order_line_vals()` es fundamental porque:

- calcula el precio según la tarifa activa,
- convierte diferencias con lista pública en discount,
- calcula `price_subtotal` y `price_subtotal_incl` sobre el precio efectivo,
- respeta posición fiscal.

Este método es reutilizado por `pos_conventional_order_barcode`, por lo que es una **pieza de dominio compartida**.

1.8.3.2.4 Validación y facturación desde backend

`action_validate_and_invoice()` hace el ciclo completo:

1. asignar cliente fallback si falta,
2. marcar `to_invoice`,
3. pasar el pedido a pagado,
4. generar factura si procede,
5. devolver la acción de siguiente paso.

1.8.3.2.5 Navegación post-validación

`_get_post_validation_action()` construye una acción “Nuevo pedido” y, si está activa la impresión automática, la envuelve en un `client action` que imprime y luego redirige.

1.8.3.3models/pos_order_line.py

Este fichero es **muy importante** aunque a menudo pase desapercibido.

Responsabilidades:

- cálculo de `total_cost`
- marca `is_total_cost_computed`
- inverse de `tax_ids_after_fiscal_position`
- salvaguarda de `tax_ids` cuando OWL no los persiste bien
- recálculo inmediato del total del pedido al tocar líneas

Hallazgo clave:

El módulo corrige problemas reales de Odoo 19 con formularios OWL y campos computados/inversos.

Código importante:

```
def _inverse_tax_ids_after_fiscal_position(self):  
    if not line.order_id.fiscal_position_id:  
        line.tax_ids = line.tax_ids_after_fiscal_position
```

y:

```
@api.onchange("qty", "discount", "price_unit", "tax_ids")  
def _onchange_qty(self):  
    result = super()._onchange_qty()  
    self._onchange_recompute_parent_order_totals()  
    return result
```

Impacto:

- evita pérdida silenciosa de impuestos,
- mantiene el total del pedido visualmente consistente,
- permite documentar coste/margen por línea.

[1.8.3.4models/res_config_settings.py](#)

Responsabilidades:

- expone pos_non_touch en ajustes
- expone cliente por defecto
- impide cambiar modo táctil/no táctil si hay sesiones abiertas

Esto protege la consistencia operativa de la caja.

1.8.4 Frontend/JS clave

[1.8.4.1static/src/js/pos_order_list_controller.js](#)

Es el controlador de la lista de pedidos convencional.

Hace varias cosas críticas:

- recuerda la sesión activa en sessionStorage,
- detecta si está dentro de una sesión no táctil,
- añade acciones "Entrada / Salida de efectivo" y "Cerrar caja",
- intercepta createRecord() para pedir PIN si está activado,
- abre sale.order si el pedido POS está vinculado a venta.

1.8.4.2 `static/src/js/pos_receipt_client_action.js`

Responsabilidad crítica:

- cargar el reporte HTML en un `iframe` oculto,
- dejar que el propio informe lance `window.print()`,
- navegar inmediatamente a la siguiente acción.

Es una implementación elegante para no bloquear el flujo de caja después de validar.

1.8.5 Assets declarados por core

Muy relevantes:

- `pos_new_order_action.js`
- `pos_order_list_controller.js`
- `pos_order_list_auto_open.js`
- `pos_action_service_patch.js`
- `product_label_section_and_note_field_patch.js`
- `pos_receipt_client_action.js`

1.8.6 Cobertura de tests

Fuerte y variada:

- `test_pos_config.py`
- `test_pos_order.py`
- `test_pos_order_completeness.py`
- `test_pos_order_pricelist.py`
- `test_pos_order_line_cost_margin.py`
- `test_receipt_print.py`
- `test_res_config_settings.py`
- tests JS/HOOT del parche de líneas

1.8.7 Conclusión del módulo

`pos_conventional_core` contiene la **semántica operativa principal** del producto. Si hay que entender una regresión de negocio, casi siempre hay que empezar aquí.

1.9 5.2 pos_conventional_session_management

1.9.1 Objetivo

Gestiona el **ciclo de vida de la caja** en modo convencional:

- apertura,
- saldo inicial,
- movimientos de entrada/salida,
- cierre,
- herencia de saldo final anterior,
- popups OWL específicos.

1.9.2 Backend clave

1.9.2.1 *models/pos_session.py*

Responsabilidades:

- `_cancel_empty_draft_orders()` antes del cierre
- override de `post_closing_cash_details()`
- override de `close_session_from_ui()`
- `get_closing_control_data_non_touch()` con datos de moneda
- heredar saldo final anterior al crear nueva sesión
- `action_pos_session_open()` para lanzar popup convencional

Hallazgo muy importante:

El método `_cancel_empty_draft_orders()` evita que los pedidos vacíos creados por el flujo de “nuevo pedido” bloqueen el cierre de caja.

Código importante:

```
empty_draft = self.env["pos.order"].search([
    ("session_id", "=", self.id),
    ("state", "=", "draft"),
    ("lines", "=", False),
])
empty_draft.write({"state": "cancel"})
```

Este detalle es crítico para operación real en tienda.

1.9.2.2 *wizard/pos_session_opening_wizard.py*

Responsabilidades:

- cargar caja inicial por defecto desde la sesión,
- contar pedidos pendientes de la configuración,
- abrir sesión desde backend,
- validar PIN indirectamente si aplica,
- volver a la lista de pedidos de esa caja.

1.9.2.3Otros wizards

- `pos_session_closing_wizard.py`: cierre y resumen
- `pos_session_cash_move_wizard.py`: entradas/salidas de efectivo
- `pos_session_closing_payment_line.py`: diferencias por método de pago

1.9.3 Frontend/OWL

1.9.3.1static/src/js/opening_popup.js

Popup de apertura:

- lee sesión/configuración/moneda,
- propone el saldo inicial,
- llama a `set_opening_control`,
- redirige luego a la lista de pedidos.

1.9.3.2static/src/js/closing_popup.js

Responsabilidades esperables por el código indexado:

- cargar datos de cierre,
- desglosar por métodos de pago,
- calcular diferencia,
- permitir cierre desde popup.

1.9.3.3static/src/js/cash_move_popup.js

Responsabilidades:

- capturar importe y tipo de movimiento,
- validar formato,
- confirmar entrada/salida de efectivo.

1.9.4 Data / configuración

- `data/pos_session_sequence.xml`
- vistas de apertura/cierre/movimientos
- assets OWL backend

1.9.5 Cobertura de tests

- `test_session_management.py`

Casos cubiertos relevantes:

- arrastre de saldo de cierre anterior,
- apertura en modo no táctil,
- wizard de apertura,
- estructura de datos del popup de cierre.

1.9.6 Conclusión del módulo

Es el **subsistema de caja** del producto. Si el proyecto se ve como “TPV convencional + caja”, este módulo es la mitad de la solución.

1.10 5.3 `pos_conventional_payment_wizard`

1.10.1 Objetivo

Implementa el **motor de cobro guiado** del POS convencional.

1.10.2 Qué aporta

- botón rápido de efectivo,
- botón rápido de tarjeta,
- popup de pago mixto,
- cálculo de importe pendiente,
- cálculo de cambio,
- soporte de pagos negativos para devoluciones/cambio,
- validación final con redirección a nuevo pedido.

1.10.3 Backend clave

1.10.3.1 `models/pos_order.py`

Métodos funcionales principales:

- `_get_previous_sale_banner_params()`
- `_is_negative_payment_flow()`
- `action_pay_cash()`
- `action_pay_card()`
- `action_pos_convention_pay_with_method()`
- `action_open_payment_popup()`

- `action_cancel_and_delete_order()`
- `get_payment_popup_data()`
- `add_payment_from_ui()`
- `remove_payment_from_ui()`
- `action_register_payments_and_validate()`

Hallazgos importantes:

1. Distingue entre flujo positivo y flujo negativo/refund.
2. El cambio se modela como **pago negativo en efectivo**.
3. El popup frontend puede operar añadiendo/eliminando pagos por RPC.

1.10.3.2 *wizard/pos_make_payment_wizard.py*

Es probablemente el wizard más delicado del repositorio.

Responsabilidades:

- cargar pedido con `sudo()` para evitar `MissingError` multicompañía,
- recalcular totales desde líneas, no solo desde campos persistidos,
- ofrecer métodos disponibles según caja/contexto,
- validar suficiencia del importe,
- registrar pagos,
- registrar cambio como pago negativo,
- asignar cliente fallback,
- marcar `to_invoice`,
- ejecutar `_process_saved_order(False)`,
- lanzar impresión y siguiente pedido.

Código importante:

```
if self.is_cash_payment and self.amount_change < -0.01:
    order.add_payment({"amount": self.amount_tendered, ...})
    order.add_payment({"amount": self.amount_change, ...}) # negativo
= cambio
```

y:

```
order._process_saved_order(False)
if order.state in {"paid", "done"}:
    order._send_order()
    order.config_id.notify_synchronisation(...)
```

Este wizard es el **corazón del cierre de venta** cuando el pago se hace desde backend.

1.10.3.3 *wizard/pos_make_payment.py*

Mantiene compatibilidad con el wizard estándar y expone acciones rápidas de pago.

1.10.4 Frontend/JS

1.10.4.1 *static/src/js/pos_payment_buttons.js*

- actualiza métodos mostrados
- gestiona click de pago
- reproduce beep de error

1.10.4.2 *static/src/js/payment_popup.js*

- popup de selección y composición de pagos
- teclado
- actualización incremental
- validación final

1.10.4.3 *static/src/js/cash_change_banner.js*

- muestra el resumen de la venta anterior
- total, cambio y tipo de pago
- útil para la operación continua en caja

1.10.5 Cobertura de tests

- *test_payment_wizard.py*
- *test_payment_flow.py*

Se verifica, entre otros:

- disponibilidad de métodos,
- apertura de wizard correcto,
- flujo tarjeta,
- estructura del popup,
- validación de importes.

1.10.6 Conclusión del módulo

Es el **núcleo de cobro**. Si falla, el sistema puede seguir navegando, pero no puede vender correctamente.

1.11 5.4 pos_conventional_order_barcode

1.11.1 Objetivo

Permite **añadir productos al pedido desde el formulario backend** mediante lector de código de barras o referencia interna (default_code).

1.11.2 Backend clave

1.11.2.1 *models/pos_order.py*

Responsabilidades:

- preparar datos de línea desde barcode,
- reutilizar `_prepare_order_line_vals()` si existe,
- recomputar totales del pedido,
- exponer RPC `get_product_line_data_by_barcode()`,
- exponer `add_product_by_barcode()`.

Hallazgos importantes:

- primero busca por barcode, después por `default_code`.
- si el producto ya está en líneas, incrementa cantidad en vez de duplicar línea.
- al crear o acumular, recalcula subtotales e impuestos.

Código importante:

```
existing_line = self.lines.filtered(lambda l: l.product_id.id ==
product.id)
if existing_line:
    new_qty = line.qty + qty_to_add
    ...
else:
    vals = self._prepare_barcode_line_vals_from_scan(product,
line_vals=line_vals)
    self.env["pos.order.line"].create(vals)
```

1.11.3 Frontend clave

1.11.3.1 *static/src/js/pos_order_form_barcode_controller.js*

Es uno de los JS más complejos del repositorio.

Responsabilidades detectadas:

- captura global de `keydown` para lector tipo keyboard wedge,

- buffer temporal de caracteres,
- diferenciación entre escritura manual y escaneo rápido,
- interceptación del botón de pago para bloquear cobros inválidos,
- beforeLeave personalizado para guardar antes de salir,
- limpieza agresiva de foco cuando se añaden líneas manuales,
- compatibilidad con navegación del formulario y clics especiales.

Hallazgo técnico relevante:

Este controlador existe porque el flujo backend de Odoo no está diseñado de forma nativa para uso intensivo con escáner + formulario editable. El módulo compensa esa carencia con bastante lógica de DOM/foco.

1.11.4 Cobertura de tests

- `test_order_barcode.py`
- `test_order_barcode_frontend.py`
- `static/tests/pos_order_form_barcode_controller.test.js`

1.11.5 Conclusión del módulo

Es la pieza que convierte el formulario backend en una herramienta operativa real para tienda física.

1.12 5.5 `pos_conventional_receipt`

1.12.1 Objetivo

Personaliza el ticket del POS clásico y garantiza presencia de datos de empresa en la exportación del pedido.

1.12.2 Backend clave

1.12.2.1 `models/pos_order.py`

Sobrescribe:

- `export_for_ui()`
- `to_json()`
- `export_as_JSON()`
- `export_as_json()`

Si faltan datos de empresa, los rellena desde `self.env.company`.

1.12.3 Assets POS

Bundle `point_of_sale._assets_pos`:

- `static/src/css/pos_receipt.scss`
- `static/src/xml/receipt_templates.xml`
- `static/src/js/receipt_order.js`

1.12.4 Observación

Es un módulo pequeño pero importante porque estabiliza la salida del ticket en el frontend POS estándar y sirve de base visual a la suite.

1.13 5.6 `pos_conventional_receipt_custom`

1.13.1 Objetivo

Amplía la capa de impresión/backend con:

- factura simplificada 80mm,
- envío por email,
- enriquecimiento de datos del recibo,
- detalle de impuestos por ticket.

1.13.2 Backend clave

1.13.2.1 `models/pos_order.py`

Métodos importantes:

- `get_factura_report_url()`
- `action_print_factura_simplificada()`
- `action_send_email()`
- `get_order_receipt_data()`
- `_get_receipt_tax_details()`

Hallazgo arquitectónico importante:

Este módulo **no reconstruye** el ticket completo; solo devuelve enriquecimientos para que `pos_conventional_core.get_order_receipt_data()` los mezcle vía `super()`.

Esto depende del orden de carga/MRO y está bien pensado.

Código importante:

```
return {  
    "company": {...},  
    "tax_details": order._get_receipt_tax_details(),  
}
```

1.13.3 Data / reports

- data/mail_template_pos_receipt.xml
- report/pos_order_report.xml

1.13.4 Cobertura de tests

- test_receipt_custom.py

1.13.5 Conclusión del módulo

Es el **módulo documental/comercial**: hace que la venta no solo se cobre, sino que deje ticket y comunicación listos.

1.14 5.7 pos_conventional_picking_integration

1.14.1 Objetivo

Permite convertir un pedido POS en un **pedido de venta tradicional** y procesar su albarán.

1.14.2 Backend clave

1.14.2.1 *models/pos_order.py*

Añade:

- estado linked
- campo linked_sale_order_id
- bandera is_linked_to_sale
- bandera show_albaran_button
- acción action_pay_account()

Flujo de action_pay_account():

1. valida que el pedido está en borrador, con líneas y cliente,
2. crea sale.order con líneas equivalentes,
3. vincula el POS con la venta,
4. confirma la venta,
5. procesa picking,

6. imprime albarán 80mm en iframe,
7. vuelve al flujo convencional con `next_action` si existe.

Código importante:

```
self.write({
    "linked_sale_order_id": sale_order.id,
    "name": sale_order.name,
    "state": "linked",
})
```

Observación importante:

El pedido POS cambia su name para alinearse con el `sale.order`. Esto es útil operativamente, pero conviene recordarlo al trazar auditoría.

1.14.3 Report

- `report/albaran_receipt.xml`

1.14.4 Cobertura de tests

- `test_picking_integration.py`

1.14.5 Conclusión del módulo

Es la puerta entre el TPV y el circuito de venta logística tradicional.

1.15 5.8 `pos_conventional_sale_integration`

1.15.1 Objetivo

Aporta integración ligera con ventas y reporting.

1.15.2 Backend clave

1.15.2.1 `models/pos_order.py`

- `open_linked_sale_order()` para navegar al pedido de venta asociado.

1.15.2.2 `models/report_sale_details.py`

Extiende el informe `report.point_of_sale.report_saledetails` añadiendo bloque `customer_account`:

- total vinculado,
- número de pedidos,
- detalle de pedidos POS enlazados a venta.

1.15.3 Cobertura de tests

- `test_sale_integration.py`

1.15.4 Conclusión del módulo

Complementa el módulo de albaranes con una capa de explotación de datos y navegación.

1.16 5.9 `pos_conventional_returns`

1.16.1 Objetivo

Implementa un flujo de devoluciones **acotado a la caja/configuración activa**.

1.16.2 Backend clave

1.16.2.1 `models/pos_order.py`

Responsabilidades:

- `_refund()` con contexto `skip_completeness_check`
- `refund()` con validación de líneas reembolsables
- `action_open_conventional_returns()`

Hallazgo importante:

La devolución no abre el universo completo de pedidos POS: lo limita a la **misma configuración/caja** y excluye borradores/cancelados.

Código importante:

```
action["domain"] = [  
    ("config_id", "=", config_id),  
    ("state", "not in", ["draft", "cancel"]),  
]
```

1.16.3 Frontend

- `static/src/js/pos_order_list_returns_patch.js`

1.16.4 Cobertura de tests

- `test_pos_conventional_returns.py`

1.16.5 Conclusión del módulo

Protege la operativa diaria haciendo que las devoluciones se ejecuten dentro del contexto correcto de caja.

1.17 5.10 pos_conventional_users_pin

1.17.1 Objetivo

Añade autenticación de usuario por PIN POS para:

- apertura de caja,
- nuevo pedido si se fuerza login tras cada venta,
- cambio de usuario tras una operación.

1.17.2 Backend clave

1.17.2.1 *models/res_users.py*

- campo pos_pin
- constraint de unicidad global del PIN

1.17.2.2 *models/pos_config.py*

- campo pos_force_employee_login_after_order
- override de `_get_non_touch_opening_action()`

Si la opción está activa, en vez de abrir directamente el popup de apertura, obliga a pasar por `pos.session.pin.wizard`.

1.17.2.3 *wizard/pos_session_pin_wizard.py*

Gestiona tres flujos:

1. cambio de usuario tras una venta,
2. creación de nuevo pedido con PIN obligatorio,
3. validación previa a apertura de sesión.

Código importante:

```
if self.env.context.get("switch_user_after_sale"):
    return {"tag": "pos_conventional_new_order", ...}
if self.env.context.get("force_new_order_flow"):
    return {"res_model": "pos.order", ...}
return {"tag": "pos_conventional_opening_popup", ...}
```

1.17.3 Cobertura de tests

- `test_users_pin.py`

1.17.4 Conclusión del módulo

Introduce trazabilidad operativa por empleado y endurece el acceso a caja.

1.18 5.11 `pos_conventional_config_user_filter`

1.18.1 Objetivo

Restringe qué cajas POS puede usar cada usuario.

1.18.2 Backend clave

1.18.2.1 `models/res_users.py`

Añade `allowed_pos_config_ids` y helpers:

- `_has_limited_pos_config_access()`
- `_get_effective_allowed_pos_config_ids()`
- `_can_access_pos_config()`

1.18.2.2 `controllers/main.py`

Extiende `PosController.pos_web()` para devolver 404 si el usuario no puede acceder a la `pos.config` solicitada.

Hallazgo importante:

La restricción no se queda solo en vistas; también baja al acceso HTTP al POS.

1.18.3 Seguridad

- `security/pos_config_record_rules.xml`
- `views/res_users_views.xml`

1.18.4 Cobertura de tests

- `test_user_filter.py`

1.18.5 Conclusión del módulo

Es el módulo de **segregación operativa** por caja/usuario.

1.19 5.12 pos_conventional_cash_calculator

1.19.1 Objetivo

Wizard de conteo manual de efectivo por denominaciones.

1.19.2 Backend clave

1.19.2.1 *wizard/cashbox_calculator_mixin.py*

- helper de cálculo total por denominaciones

1.19.2.2 *wizard/pos_cash_calculator_wizard.py*

Responsabilidades:

- campos por billete/moneda,
- cálculo del total,
- integración con wizard padre de cierre o movimiento de caja,
- botones de incremento/decremento rápidos.

Hallazgo:

No es un simple helper aislado; está integrado para escribir de vuelta en:

- `pos.session.closing.wizard`
- `pos.session.cash_move.wizard`

1.19.3 Cobertura de tests

- `test_cashbox_calculator.py`
- `test_pos_cash_calculator_wizard.py`

1.19.4 Conclusión del módulo

Mejora usabilidad y exactitud en apertura/cierre/movimientos de caja.

1.20 5.13 pos_conventional_cash_drawer

1.20.1 Objetivo

Abre el cajón de efectivo desde el flujo convencional.

1.20.2 Backend clave

1.20.2.1 *models/pos_order.py*

- `action_open_cash_drawer_from_conventional()` delega en `config_id.action_test_cash_drawer()`.

1.20.3 Frontend

1.20.3.1 *static/src/js/pos_payment_buttons_cash_drawer.js*

Amplía botones de pago con botón de apertura de cajón.

1.20.4 Dependencia externa relevante

- `xtendoo_cash_drawer`

1.20.5 Cobertura de tests

- `test_pos_conventional_cash_drawer.py`

1.20.6 Conclusión del módulo

Pequeño, pero importante para la operación física de caja.

1.21 6. Flujos funcionales clave

1.22 6.1 Apertura de caja no táctil

Usuario abre TPV

- ``pos.config.open_ui()``
- crea/recupera ``pos.session``
- si ``opening_control``:
 - con PIN activado: ``pos.session.pin.wizard``
 - sin PIN: ``pos_conventional_opening_popup``
- apertura confirmada
- redirección a lista de ``pos.order`` de la caja

Puntos de código:

- `pos_conventional_core/models/pos_config.py`
- `pos_conventional_users_pin/models/pos_config.py`
- `pos_conventional_session_management/models/pos_session.py`
- `pos_conventional_session_management/static/src/js/opening_popup.js`

1.23 6.2 Creación de pedido backend convencional

Lista de pedidos

- Nuevo pedido

- ``default_get()`` asigna sesión/tarifa/cliente por defecto
- formulario editable
- validación de completitud en borrador

Puntos de código:

- `pos_conventional_core/models/pos_order.py`
- `pos_conventional_core/static/src/js/pos_order_list_controller.js`

1.24 6.3 Alta de líneas por barcode

Escáner emite teclas rápidas

- controlador JS acumula buffer
- RPC a búsqueda por barcode/default_code
- si existe línea, aumenta qty
- si no existe, crea línea
- recalcula totales del pedido

Puntos de código:

- `pos_conventional_order_barcode/static/src/js/pos_order_form_barcode_controller.js`
- `pos_conventional_order_barcode/models/pos_order.py`
- `pos_conventional_core/models/pos_order.py`

1.25 6.4 Reprecio al cambiar cliente

Cambio de partner

- obtiene pricelist del partner o la de sesión
- reasigna ``pricelist_id``
- recorre líneas
- recalcula ``price_unit``, ``discount``, subtotales
- recalcula total del pedido

Punto central:

- `pos_conventional_core/models/pos_order.py::_onchange_partner_id_update_pricelist`

1.26 6.5 Cobro en efectivo / tarjeta / mixto

Pedido válido

- botón de pago rápido o popup
- wizard calcula pendiente
- registra pagos
- si efectivo y sobra dinero, registra cambio como pago negativo
- asigna cliente fallback si falta
- marca ``to_invoice``
- ``_process_saved_order(False)``
- imprime si corresponde
- abre siguiente pedido

Puntos de código:

- `pos_conventional_payment_wizard/models/pos_order.py`
- `pos_conventional_payment_wizard/wizard/pos_make_payment_wizard.py`
- `pos_conventional_core/static/src/js/pos_receipt_client_action.js`

1.27 6.6 Venta a cuenta / albarán

Pedido POS borrador con cliente

- ``action_pay_account()``
- crea ``sale.order``
- confirma venta y picking
- imprime albarán 80mm
- redirige al siguiente pedido

Puntos de código:

- `pos_conventional_picking_integration/models/pos_order.py`

1.28 6.7 Devoluciones

Desde lista convencional

- acción de devoluciones
- se filtran pedidos de la misma caja
- se ejecuta refund
- se evita bloqueo por constraint de completitud

Puntos de código:

- `pos_conventional_returns/models/pos_order.py`

1.29 6.8 Cierre de caja

Lista de pedidos

- acción "Cerrar caja"
- popup OWL de cierre
- cancela pedidos vacíos en borrador
- calcula diferencias
- cierra sesión

Puntos de código:

- `pos_conventional_core/static/src/js/pos_order_list_controller.js`
 - `pos_conventional_session_management/models/pos_session.py`
 - `pos_conventional_session_management/static/src/js/closing_popup.js`
-

1.30 7. Mapa de código importante

Área	Fichero	Motivo
Entrada al modo convencional	pos_conventional_core/models/pos_config.py	Decide si abrir POS clásico o flujo no táctil
Compleitud de pedido	pos_conventional_core/models/pos_order.py	Evita borradores inválidos
Precios/tarifas	pos_conventional_core/models/pos_order.py	Reprecio por partner y pricelist
Coste e impuestos línea	pos_conventional_core/models/pos_order_line.py	Corrige problemas OWL y calcula coste total
Lista operacional	pos_conventional_core/static/src/js/pos_order_list_controller.js	Menú de caja, PIN y navegación
Impresión no bloqueante	pos_conventional_core/static/src/js/pos_receipt_client_action.js	Imprime y continúa flujo
Motor de pago	pos_conventional_payment_wizard/wizard/pos_make_payment_wizard.py	Registra pagos y valida venta
API de pagos popup	pos_conventional_payment_wizard/models/pos_order.py	Backend RPC para popup JS
Escaneo de productos	pos_conventional_order_barcode/static/src/js/pos_order_form_barcode_controller.js	Captura barcode desde formulario
Alta por barcode	pos_conventional_order_barcode/models/pos_order.py	Busca y crea/acumula líneas
Apertura/cierre de	pos_conventional_session_management/	Ciclo operativo de

Área	Fichero	Motivo
caja	models/ pos_session.py	caja
Popup de apertura	pos_conventional_session_management/ static/src/js/ opening_popup.js	UX de apertura
Factura simplificada	pos_conventional_receipt_custom/ models/ pos_order.py	URL, impresión, email, tax details
Venta/albarán	pos_conventional_picking_integration/ models/ pos_order.py	Puente TPV → venta/logística
Devoluciones	pos_conventional_returns/models/ pos_order.py	Scope correcto por caja
PIN POS	pos_conventional_users_pin/wizard/ pos_session_pin_wizard.py	Tres flujos de autenticación
Seguridad por caja	pos_conventional_config_user_filter/ controllers/ main.py	Restringe acceso HTTP al POS

1.31 8. Cobertura de tests existente

1.31.1 Cobertura backend/frontend detectada

Módulo	Tests relevantes
pos_conventional_cash_calculator	cálculo de efectivo y wizard
pos_conventional_cash_drawer	apertura de cajón y assets
pos_conventional_config_user_filter	permisos y cajas permitidas
pos_conventional_core	config, pedido, completitud, tarifas, costes, impresión, ajustes, JS patch
pos_conventional_order_barcode	búsqueda por barcode/default_code, alta de

Módulo	Tests relevantes
	líneas, frontend
pos_conventional_payment_wizard	flujo de pago y popup
pos_conventional_picking_integration	creación de sale.order / albarán
pos_conventional_receipt_custom	URL, email, datos extra de ticket
pos_conventional_returns	dominio de devoluciones y refund
pos_conventional_sale_integration	report_saledetails extendido
pos_conventional_session_management	apertura/cierre y saldo heredado
pos_conventional_users_pin	unicidad PIN y flujos de autenticación

1.31.2 Valoración

La suite tiene una cobertura bastante buena para ser una personalización compleja de Odoo:

- hay pruebas de backend funcional,
- hay pruebas de frontend/HOOT en puntos delicados,
- se cubren regresiones reales del negocio.

1.31.3 Huecos potenciales a vigilar

- interacción simultánea entre barcode + payment popup + beforeLeave,
 - escenarios multicompañía con permisos restringidos,
 - cadena completa PIN -> nuevo pedido -> pago -> impresión -> siguiente pedido,
 - impresión/iframe según navegador.
-

1.32 9. Riesgos técnicos y observaciones de mantenimiento

1.33 9.1 Dependencia del orden de carga

La construcción del ticket enriquecido depende de la cadena `super()` y del rol agregador de `pos_conventional_core`. Si se altera la jerarquía o las dependencias de manifiesto, puede romperse la composición del recibo.

1.34 9.2 Mucha lógica de frontend basada en DOM

Especialmente en `pos_conventional_order_barcode/static/src/js/pos_order_form_barcode_controller.js`. Cualquier cambio fuerte en clases CSS o estructura OWL/list/form de Odoo puede afectar.

1.35 9.3 Sensibilidad multicompañía

Hay varios `sudo()` y defensas frente a `MissingError`. Están puestos por una razón: no deben retirarse “por limpieza” sin reprobador escenarios reales.

1.36 9.4 Cierre de caja y borradores vacíos

La cancelación previa de pedidos vacíos es funcionalmente esencial. Si se toca el flujo de “nuevo pedido”, hay que revisar el cierre.

1.37 9.5 Pago y cambio

El cambio se representa como pago negativo. Cualquier integración externa o reporte que lea `pos.payment` debe entender esta convención.

1.38 10. Recomendaciones prácticas para futuras modificaciones

1.39 10.1 Archivo que debe mantenerse siempre actualizado

Archivo canónico: `DOCUMENTACION_TECNICA_POS_CONVENTIONAL.md`

Regla recomendada:

- cualquier cambio funcional en un módulo de esta suite debe reflejarse aquí,
- el PDF/DOCX deben regenerarse después desde este Markdown.

1.40 10.2 Cuándo actualizar esta documentación

Actualizar obligatoriamente cuando cambie cualquiera de estos puntos:

- manifiestos/dependencias,
- acciones de apertura/cierre,
- flujo de pago,
- impresión/recibos,
- comportamiento de barcode,
- integración con `sale.order/stock.picking`,
- seguridad/PIN/record rules,
- tests significativos.

1.41 10.3 Formato recomendado de actualización

Para cada cambio futuro, añadir o ajustar:

1. módulo afectado,
 2. descripción funcional,
 3. ficheros tocados,
 4. impacto en flujos,
 5. impacto en tests,
 6. si aplica, snippet de código importante.
-

1.42 11. Comandos de regeneración del entregable

Usar el script del repositorio:

```
cd /home/xtendoo/Escritorio/odoo/odoo_19/odoo/custom/src/xtendoo-pos-conventional
./exportar_documentacion_pos_conventional.sh
```

Salida esperada:

- DOCUMENTACION_TECNICA_POS_CONVENTIONAL.docx
 - DOCUMENTACION_TECNICA_POS_CONVENTIONAL.pdf
-

1.43 12. Conclusión final

La suite `xtendoo-pos-conventional` no es una customización aislada, sino una **plataforma de TPV convencional completa** sobre Odoo 19. Su diseño resuelve problemas reales de operación en tienda desde backend:

- caja no táctil,
- navegación rápida entre ventas,
- escaneo por teclado,
- cobro guiado,
- impresión inmediata,
- control de efectivo,
- trazabilidad por usuario,
- integración con venta/logística.

Las piezas más sensibles y estratégicas del repositorio son:

1. `pos_conventional_core`
2. `pos_conventional_payment_wizard`
3. `pos_conventional_session_management`
4. `pos_conventional_order_barcode`
5. `pos_conventional_receipt_custom`

Si en el futuro hay que priorizar revisiones, auditorías o refactors, empezaría por ese orden.

1.44 13. Apéndice: índice rápido de tests por módulo

`pos_conventional_cash_calculator`

- `tests/test_cashbox_calculator.py`
- `tests/test_pos_cash_calculator_wizard.py`

`pos_conventional_cash_drawer`

- `tests/test_pos_conventional_cash_drawer.py`

`pos_conventional_config_user_filter`

- `tests/test_user_filter.py`

`pos_conventional_core`

- `tests/test_pos_config.py`
- `tests/test_pos_order.py`

- tests/test_pos_order_completeness.py
- tests/test_pos_order_pricelist.py
- tests/test_pos_order_line_cost_margin.py
- tests/test_receipt_print.py
- tests/test_res_config_settings.py
- tests/test_product_label_section_and_note_field_js.py
- static/tests/product_label_section_and_note_field_patch.test.js

pos_conventional_order_barcode

- tests/test_order_barcode.py
- tests/test_order_barcode_frontend.py
- static/tests/pos_order_form_barcode_controller.test.js

pos_conventional_payment_wizard

- tests/test_payment_wizard.py
- tests/test_payment_flow.py

pos_conventional_picking_integration

- tests/test_picking_integration.py

pos_conventional_receipt_custom

- tests/test_receipt_custom.py

pos_conventional_returns

- tests/test_pos_conventional_returns.py

pos_conventional_sale_integration

- tests/test_sale_integration.py

pos_conventional_session_management

- tests/test_session_management.py

pos_conventional_users_pin

- tests/test_users_pin.py